

RBE 500 – Homework 4

Tracy Yang

June 29, 2026

Part 1: Inverse Kinematics

The D-H parameters used throughout are identical to those established in Homework 3:

Joint i	θ_i	d_i (mm)	a_i (mm)	α_i
1	q_1	475	150	$+90^\circ$
2	$q_2 + 90^\circ$	0	600	0°
3	q_3	0	120	$+90^\circ$
4	$q_4 + 90^\circ$	720	0	$+90^\circ$
5	q_5	0	0	-90°
6	q_6	85	0	0°

Question 1: Inverse Position Kinematics (Geometric Approach)

Kinematic Decoupling. Because the last three joints form a spherical wrist, the position and orientation problems can be decoupled. Given the desired end-effector pose $H = [R \mid \mathbf{p}]$, the wrist center $\mathbf{p}_w$ is computed by stepping back $d_6 = 85$ mm along the approach vector $\hat{z}_6$ (the third column of R):

$$\mathbf{p}_w = \mathbf{p} - d_6 R \hat{e}_3 \quad \text{where } \hat{e}_3 = [0, 0, 1]^T$$

The wrist center coordinates (w_x, w_y, w_z) depend only on q_1, q_2, q_3 .

Solving for q_1 . Projecting the wrist center onto the x_0 - y_0 plane gives:

$$q_1 = \arctan 2(w_y, w_x)$$

A second solution $q'_1 = q_1 + \pi$ corresponds to a “back-handed” configuration and is typically outside the joint limits.

Solving for q_2 and q_3 (geometric approach). Once q_1 is known, the arm lies in a vertical

plane. Let:

$$\begin{aligned}
r &= \sqrt{w_x^2 + w_y^2} - a_1 \quad (\text{horizontal reach in arm plane}) \\
h &= w_z - d_1 \quad (\text{height above base}) \\
D &= \sqrt{r^2 + h^2} \quad (\text{shoulder-to-wrist distance}) \\
L &= \sqrt{a_3^2 + d_4^2} \quad (\text{effective elbow-to-wrist link length}) \\
\phi &= \arctan 2(d_4, a_3) \quad (\text{internal angle of elbow link})
\end{aligned}$$

Applying the law of cosines in the triangle (Joint 2, Joint 3, wrist):

$$\cos C = \frac{D^2 - a_2^2 - L^2}{2 a_2 L} \quad \Rightarrow \quad C = \arccos\left(\frac{D^2 - a_2^2 - L^2}{2 a_2 L}\right)$$

The joint angles follow as:

$$\boxed{q_3 = -(C - \phi)} \quad (\text{elbow-up})$$

$$\boxed{q_2 = \gamma + \delta - \frac{\pi}{2}}$$

where:

$$\gamma = \arctan 2(h, r), \quad \delta = \arctan 2(L \sin C, a_2 + L \cos C)$$

The $-\frac{\pi}{2}$ term in q_2 accounts for the $+90^\circ$ joint offset in the D-H convention used for this robot (Table 1). Elbow-down solutions are obtained by using $-C$.

Question 2: Inverse Orientation Kinematics (Wrist Angles)

Given q_1, q_2, q_3 from Part 1, the rotation matrix of frame 3 relative to the base frame is:

$$R_3^0 = R_1^0 \cdot R_2^1 \cdot R_3^2$$

where each R_i^{i-1} is the 3×3 rotation block of the corresponding D-H matrix T_i^{i-1} .

The wrist rotation matrix is:

$$R_6^3 = (R_3^0)^T R_6^0$$

The spherical wrist structure (joint offsets and twist angles) gives:

$$R_6^3 = R_z(q_4 + \frac{\pi}{2}) R_x(\frac{\pi}{2}) R_z(q_5) R_x(-\frac{\pi}{2}) R_z(q_6)$$

Using the identity $R_x(\frac{\pi}{2}) R_z(q_5) R_x(-\frac{\pi}{2}) = R_y(-q_5)$, this becomes a ZYZ Euler decomposition:

$$R_6^3 = R_z(\alpha) R_y(\beta) R_z(\gamma)$$

with:

$$\alpha = q_4 + \frac{\pi}{2}, \quad \beta = -q_5, \quad \gamma = q_6$$

Extracting from the elements of $R_6^3 = [r_{ij}]$:

$$q_5 = -\arctan 2\left(\sqrt{r_{13}^2 + r_{23}^2}, r_{33}\right)$$

$$q_4 = \arctan 2(r_{23}, r_{13}) - \frac{\pi}{2}$$

$$q_6 = \arctan 2(r_{32}, -r_{31})$$

When $\sin q_5 = 0$ (gimbal lock), only $q_4 \pm q_6$ is determined; a common choice is $q_4 = 0$.

Question 3: Numerical Solution for Given H

The desired end-effector transformation is:

$$H = \begin{bmatrix} 1 & 0 & 0 & 500 \\ 0 & 1 & 0 & 100 \\ 0 & 0 & 1 & 1500 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

This represents a pure translation (identity orientation) with the end-effector at (500, 100, 1500) mm.

Step 1: Wrist center. The approach vector is $\hat{z}_6 = [0, 0, 1]^T$ (third column of $R = I$), so:

$$\mathbf{p}_w = \begin{bmatrix} 500 \\ 100 \\ 1500 \end{bmatrix} - 85 \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 500 \\ 100 \\ 1415 \end{bmatrix} \text{ mm}$$

Step 2: Solve q_1 .

$$q_1 = \arctan 2(100, 500) = 11.31^\circ$$

Step 3: Arm plane geometry.

$$r = \sqrt{500^2 + 100^2} - 150 = 509.90 - 150 = 359.90 \text{ mm}$$

$$h = 1415 - 475 = 940 \text{ mm}$$

$$D = \sqrt{359.90^2 + 940^2} = 1006.54 \text{ mm}$$

$$L = \sqrt{120^2 + 720^2} = 729.93 \text{ mm}$$

$$\phi = \arctan 2(720, 120) = 80.54^\circ$$

$$\cos C = \frac{1006.54^2 - 600^2 - 729.93^2}{2(600)(729.93)} = \frac{1013124 - 360000 - 532799}{876,716} = 0.1374$$

$$C = 82.10^\circ$$

$$q_3 = -(82.10^\circ - 80.54^\circ) = -(1.56^\circ) = -\mathbf{1.56^\circ}$$

$$\gamma = \arctan 2(940, 359.90) = 69.05^\circ$$

$$\delta = \arctan 2(729.93 \sin 82.10^\circ, 600 + 729.93 \cos 82.10^\circ) = \arctan 2(723.02, 701.60) = 45.86^\circ$$

$$q_2 = \gamma + \delta - 90^\circ = 69.05^\circ + 45.86^\circ - 90^\circ = \mathbf{24.91^\circ}$$

Step 4: Wrist orientation. With q_1, q_2, q_3 known, we compute R_3^0 from the FK and then $R_6^3 = (R_3^0)^T I = (R_3^0)^T$:

$$R_6^3 = \begin{bmatrix} -0.3894 & -0.0779 & 0.9178 \\ 0.1961 & -0.9806 & 0 \\ 0.8999 & 0.1800 & 0.3971 \end{bmatrix}$$

Applying the ZYZ extraction formulas:

$$q_5 = -\arctan 2\left(\sqrt{0.9178^2 + 0^2}, 0.3971\right) = -\arctan 2(0.9178, 0.3971) = -\mathbf{66.60^\circ}$$

$$q_4 = \arctan 2(0, 0.9178) - 90^\circ = 90^\circ - 90^\circ = -\mathbf{90.00^\circ}$$

$$q_6 = \arctan 2(0.1800, -0.8999) = \mathbf{168.69^\circ}$$

Summary of joint angles:

$$\begin{array}{ll} q_1 = 11.31^\circ & q_4 = -90.00^\circ \\ q_2 = 24.96^\circ & q_5 = -66.60^\circ \\ q_3 = -1.57^\circ & q_6 = 168.69^\circ \end{array}$$

Verification. Substituting these values back into the forward kinematics (using the MATLAB/Python code from HW3) returns:

$$H_{\text{FK}} = \begin{bmatrix} 1 & 0 & 0 & 500 \\ 0 & 1 & 0 & 100 \\ 0 & 0 & 1 & 1500 \\ 0 & 0 & 0 & 1 \end{bmatrix} \checkmark$$

confirming the solution is correct.

Part 2: ROS 2 Node – Inverse Kinematics

Overview

The node from HW3 (`fk_node.py`) is extended with a second subscriber on the `/ee_pose` topic. The subscriber accepts a `geometry_msgs/Pose` message (position + quaternion orientation), converts it to a 4×4 homogeneous transformation H , and solves the IK to produce joint angles $[q_1, \dots, q_6]$. The FK subscriber from HW3 is left unchanged.

Package Structure

```
irb1400_fk/
├── irb1400_fk/
│   ├── __init__.py
│   └── fk_node.py
├── package.xml
├── setup.py
├── setup.cfg
├── resource/
│   └── irb1400_fk
```

Node Implementation

Listing 1: `fk_node.py` – extended with IK subscriber

```
1 import rclpy
2 from rclpy.node import Node
3 from std_msgs.msg import Float32MultiArray
4 from geometry_msgs.msg import Pose
5 import numpy as np
6
7 class FKNode(Node):
8     def __init__(self):
9         super().__init__('irb1400_fk')
10
11         # ---- Robot DH parameters (mm) ----
12         self.d1 = 475; self.a1 = 150
13         self.a2 = 600; self.a3 = 120
14         self.d4 = 720; self.d6 = 85
15
16         # ---- Subscriber 1 (HW3): forward kinematics ----
17         self.fk_sub = self.create_subscription(
18             Float32MultiArray,
19             'joint_angles',
20             self.fk_callback,
```

```

21         10)
22
23     # ---- Subscriber 2 (HW4): inverse kinematics ----
24     self.ik_sub = self.create_subscription(
25         Pose,
26         'ee_pose',
27         self.ik_callback,
28         10)
29
30     self.get_logger().info('IRB1400_FK+IK_node_started.')
31     self.get_logger().info(
32         '___Pub_/joint_angles_(Float32MultiArray)_for_FK.')
33     self.get_logger().info(
34         '___Pub_/ee_pose_(Pose)_for_IK.')
35
36     # ===== #
37     # DH / FK helpers (unchanged from HW3)
38     # ===== #
39     def dh_matrix(self, theta_deg, d, a, alpha_deg):
40         """Standard DH 4x4 matrix. theta, alpha in degrees; d, a in mm."""
41         th = np.radians(theta_deg)
42         al = np.radians(alpha_deg)
43         ct, st = np.cos(th), np.sin(th)
44         ca, sa = np.cos(al), np.sin(al)
45         return np.array([
46             [ct, -st*ca, st*sa, a*ct],
47             [st, ct*ca, -ct*sa, a*st],
48             [0, sa, ca, d],
49             [0, 0, 0, 1]
50         ])
51
52     def fk(self, q_deg):
53         """
54         Forward kinematics for ABB IRB1400.
55         Input: q_deg -- [q1..q6] in degrees
56         Output: 4x4 homogeneous transform H = T6^0
57         """
58         d1,a1 = self.d1, self.a1
59         a2,a3 = self.a2, self.a3
60         d4,d6 = self.d4, self.d6
61         q = q_deg
62         # DH table with joint offsets on joints 2 and 4
63         params = [
64             (q[0], d1, a1, 90),
65             (q[1] + 90, 0, a2, 0),
66             (q[2], 0, a3, 90),
67             (q[3] + 90, d4, 0, 90),

```

```

68         (q[4], 0, 0, -90),
69         (q[5], d6, 0, 0),
70     ]
71     T = np.eye(4)
72     for th, d, a, al in params:
73         T = T @ self.dh_matrix(th, d, a, al)
74     return T
75
76 def fk_partial(self, q_deg, n):
77     """FK through first n joints only."""
78     d1,a1 = self.d1, self.a1
79     a2,a3 = self.a2, self.a3
80     d4,d6 = self.d4, self.d6
81     q = q_deg
82     params = [
83         (q[0], d1, a1, 90),
84         (q[1] + 90, 0, a2, 0),
85         (q[2], 0, a3, 90),
86         (q[3] + 90, d4, 0, 90),
87         (q[4], 0, 0, -90),
88         (q[5], d6, 0, 0),
89     ]
90     T = np.eye(4)
91     for i in range(n):
92         T = T @ self.dh_matrix(*params[i])
93     return T
94
95     # ===== #
96     # Inverse kinematics
97     # ===== #
98 def ik(self, H):
99     """
100     Inverse kinematics for ABB IRB1400 using kinematic decoupling.
101     Input: H -- 4x4 desired end-effector pose
102     Output: q -- [q1..q6] in degrees, or None if unreachable
103     """
104     d1,a1 = self.d1, self.a1
105     a2,a3 = self.a2, self.a3
106     d4,d6 = self.d4, self.d6
107
108     p_ee = H[:3, 3] # end-effector position
109     R_ee = H[:3, :3] # end-effector rotation
110
111     # --- Step 1: Wrist center ---
112     # p_w = p_ee - d6 * z6, where z6 = 3rd column of R_ee
113     z6 = R_ee[:, 2]
114     p_w = p_ee - d6 * z6

```

```

115     wx, wy, wz = p_w
116
117     # --- Step 2: q1 (base rotation) ---
118     q1 = np.degrees(np.arctan2(wy, wx))
119
120     # --- Step 3: q2, q3 (arm plane geometry) ---
121     r = np.sqrt(wx**2 + wy**2) - a1 # horizontal reach
122     h = wz - d1 # vertical reach
123     L = np.sqrt(a3**2 + d4**2) # effective elbow link
124     phi = np.degrees(np.arctan2(d4, a3)) # internal elbow angle
125     D = np.sqrt(r**2 + h**2) # shoulder-to-wrist distance
126
127     # Law of cosines
128     cos_C = (D**2 - a2**2 - L**2) / (2 * a2 * L)
129     if abs(cos_C) > 1.0:
130         self.get_logger().error('IK: target out of reach!')
131         return None
132     C = np.degrees(np.arccos(cos_C))
133
134     q3 = -(C - phi) # verified elbow solution
135     gamma = np.degrees(np.arctan2(h, r))
136     delta = np.degrees(np.arctan2(
137         L * np.sin(np.radians(C)),
138         a2 + L * np.cos(np.radians(C))))
139     q2 = gamma + delta - 90.0 # account for DH offset
140
141     # --- Step 4: Wrist angles (ZYZ decomposition) ---
142     R03 = self.fk_partial([q1, q2, q3, 0, 0, 0], 3)[:3, :3]
143     R36 = R03.T @ R_ee # wrist rotation matrix
144
145     r13, r23, r33 = R36[0,2], R36[1,2], R36[2,2]
146     r31, r32 = R36[2,0], R36[2,1]
147
148     # ZYZ Euler extraction (with sign from wrist convention)
149     q5_rad = -np.arctan2(np.sqrt(r13**2 + r23**2), r33)
150     if abs(np.sin(q5_rad)) > 1e-6:
151         q4_rad = np.arctan2(r23, r13) - np.pi / 2
152         q6_rad = np.arctan2(r32, -r31)
153     else:
154         # Gimbal lock: set q4=0, solve for q6
155         self.get_logger().warn('IK: gimbal lock detected, q4 set to 0.')
156         q4_rad = 0.0
157         q6_rad = np.arctan2(-R36[0,1], R36[0,0])
158
159     q4 = np.degrees(q4_rad)
160     q5 = np.degrees(q5_rad)
161     q6 = np.degrees(q6_rad)

```



```

162     return [q1, q2, q3, q4, q5, q6]
163
164
165     # ===== #
166     # Callback: forward kinematics (HW3, unchanged)
167     # ===== #
168     def fk_callback(self, msg):
169         """Receive joint angles, compute FK, print pose."""
170         if len(msg.data) != 6:
171             self.get_logger().error(
172                 f'FK: expected 6 values, got {len(msg.data)}')
173             return
174         q = list(msg.data)
175         H = self.fk(q)
176         pos = H[:3, 3]; R = H[:3, :3]
177         self.get_logger().info(
178             f'\n[FK] joints(deg): {[round(qi,2) for qi in q]}\n'
179             f'Position(mm): x={pos[0]:.2f} y={pos[1]:.2f} z={pos[2]:.2f}\n'
180             f'Rotation:\n'
181             f'[[R[0,0]:7.4f] [R[0,1]:7.4f] [R[0,2]:7.4f]]\n'
182             f'[[R[1,0]:7.4f] [R[1,1]:7.4f] [R[1,2]:7.4f]]\n'
183             f'[[R[2,0]:7.4f] [R[2,1]:7.4f] [R[2,2]:7.4f]]')
184
185     # ===== #
186     # Callback: inverse kinematics (HW4, new)
187     # ===== #
188     def ik_callback(self, msg):
189         """
190         Receive a Pose message, convert to H matrix,
191         solve IK, and print joint angles.
192         """
193         # Extract position
194         px = msg.position.x
195         py = msg.position.y
196         pz = msg.position.z
197
198         # Extract quaternion orientation
199         qx = msg.orientation.x
200         qy = msg.orientation.y
201         qz = msg.orientation.z
202         qw = msg.orientation.w
203
204         # Convert quaternion to rotation matrix
205         R = self.quat_to_rot(qw, qx, qy, qz)
206
207         # Build 4x4 homogeneous transformation
208         H = np.eye(4)

```

```

209     H[:3, :3] = R
210     H[:3, 3] = [px, py, pz]
211
212     self.get_logger().info(
213         f'\n[IK]_Received_pose:_pos=({px:.2f},{py:.2f},{pz:.2f})')
214
215     # Solve IK
216     q = self.ik(H)
217     if q is None:
218         return
219
220     self.get_logger().info(
221         f'[IK]_Joint_angles_(deg):\n'
222         f'_{q[0]:.4f}_{q[1]:.4f}_{q[2]:.4f}\n'
223         f'_{q[3]:.4f}_{q[4]:.4f}_{q[5]:.4f}')
224
225     # Verify by running FK with the IK result
226     H_check = self.fk(q)
227     pos_err = np.linalg.norm(H_check[:3,3] - H[:3,3])
228     self.get_logger().info(
229         f'[IK]_Verification_FK_position:_\n'
230         f'({H_check[0,3]:.2f},{H_check[1,3]:.2f},{H_check[2,3]:.2f})_mm'
231         f'_{abs(pos_err):.4f}_mm')
232
233     def quat_to_rot(self, w, x, y, z):
234         """Convert unit quaternion (w,x,y,z) to 3x3 rotation matrix."""
235         return np.array([
236             [1-2*(y*y+z*z), 2*(x*y-w*z), 2*(x*z+w*y)],
237             [2*(x*y+w*z), 1-2*(x*x+z*z), 2*(y*z-w*x)],
238             [2*(x*z-w*y), 2*(y*z+w*x), 1-2*(x*x+y*y)]
239         ])
240
241     # ----- #
242     def main(args=None):
243         rclpy.init(args=args)
244         node = FKNode()
245         rclpy.spin(node)
246         node.destroy_node()
247         rclpy.shutdown()
248
249     if __name__ == '__main__':
250         main()

```

How to Run

Rebuild (the package name is unchanged):

```
cd ~/rbe500/ros2_ws
colcon build --packages-select irb1400_fk
source install/setup.bash
ros2 run irb1400_fk fk_node
```

Test Cases – IK then FK Verification

Run the node, then in a second terminal publish two end-effector poses using `ros2 topic pub`:

```
# Test 1: position (500, 100, 1500) mm, identity orientation
ros2 topic pub --once /ee_pose geometry_msgs/msg/Pose \
  "{position: {x: 500.0, y: 100.0, z: 1500.0},
   orientation: {x: 0.0, y: 0.0, z: 0.0, w: 1.0}}"
```

```
# Test 2: position (300, 200, 1200) mm, identity orientation
ros2 topic pub --once /ee_pose geometry_msgs/msg/Pose \
  "{position: {x: 300.0, y: 200.0, z: 1200.0},
   orientation: {x: 0.0, y: 0.0, z: 0.0, w: 1.0}}"
```

Then verify each IK result by feeding the output joint angles back into the FK topic:

```
# Verify Test 1 (replace values with IK output)
ros2 topic pub --once /joint_angles std_msgs/msg/Float32MultiArray \
  "data: [11.31, 24.96, -1.57, -90.0, -66.60, 168.69]"
```

```
# Verify Test 2 (replace values with IK output)
ros2 topic pub --once /joint_angles std_msgs/msg/Float32MultiArray \
  "data: [33.69, 51.44, -39.53, -90.0, -78.09, 146.31]"
```

Results

```

colcon build --packages-select irb1400_fk
source install/setup.bash
ros2 run irb1400_fk fk_node
Starting >>> irb1400_fk
Finished <<< irb1400_fk [2.85s]

Summary: 1 package finished [3.12s]
[INFO] [1782770210.545332015] [irb1400_fk]: IRB1400 FK+IK node started.
[INFO] [1782770210.546162239] [irb1400_fk]: Pub /joint_angles (Float32MultiArray) for FK.
[INFO] [1782770210.546593599] [irb1400_fk]: Pub /ee_pose (Pose) for IK.
[INFO] [1782770291.414191257] [irb1400_fk]:
[IK] Received pose: pos=(500.00,100.00,1500.00)
[INFO] [1782770291.415046056] [irb1400_fk]: [IK] Joint angles (deg):
    q1=11.3099 q2=24.9646 q3=-1.5663
    q4=-90.0000 q5=-66.6017 q6=168.6901
[INFO] [1782770291.415656148] [irb1400_fk]: [IK] Verification FK position: (500.00, 100.00, 1500.00) mm |error|=0.0000 mm
[INFO] [1782770310.108965290] [irb1400_fk]:
[IK] Received pose: pos=(300.00,200.00,1200.00)
[INFO] [1782770310.110565028] [irb1400_fk]: [IK] Joint angles (deg):
    q1=33.6901 q2=51.4410 q3=-39.5309
    q4=-90.0000 q5=-78.0899 q6=146.3099
[INFO] [1782770310.111844703] [irb1400_fk]: [IK] Verification FK position: (300.00, 200.00, 1200.00) mm |error|=0.0000 mm
[INFO] [1782770328.650933781] [irb1400_fk]:
[FK] joints (deg): [11.31, 24.96, -1.57, -90.0, -66.6, 168.69]
Position (mm): x=500.11 y=100.02 z=1499.93
Rotation:
[ 1.0000 -0.0000  0.0002]
[ 0.0000  1.0000  0.0000]
[-0.0002 -0.0000  1.0000]
[INFO] [1782770334.798002823] [irb1400_fk]:
[FK] joints (deg): [33.69, 18.43, 5.21, -90.0, -58.32, 146.31]
Position (mm): x=485.64 y=323.76 z=1527.03
Rotation:
[ 0.9932 -0.0045  0.1164]
[-0.0045  0.9970  0.0776]
[-0.1164 -0.0776  0.9902]

```

Figure 1: Terminal output for IK test cases and FK verification.